

Guía de laboratorio de la semana 6

Spring Web y arquitectura MVC

En este laboratorio incorporaremos controladores y rutas al proyecto ejecutable de la semana 5. El producto será un catálogo consultable desde el navegador, con búsqueda por título y respuestas HTTP verificables. El estudiante deberá explicar cómo llega cada solicitud a su controlador y por qué la lógica de consulta pertenece a un servicio.

Alcance de esta práctica

Usaremos Spring MVC para navegar y responder solicitudes JSON. La interfaz seguirá siendo HTML estático con JavaScript.

1. Logro y organización del laboratorio

Al finalizar, el estudiante implementa rutas de navegación y consulta con Spring Web MVC, separa transporte HTTP de la lógica de aplicación, interpreta parámetros y estados HTTP, integra la respuesta del servidor con una interfaz accesible y demuestra el comportamiento mediante pruebas.

Criterios observables

- La aplicación sirve el APF1 sin regresiones y expone el catálogo.
- El controlador delega las consultas en CourseService por constructor.
- GET devuelve datos; parámetros inválidos reciben 400 y un ID ausente recibe 404.
- La búsqueda no depende de una lista duplicada en JavaScript.
- La interfaz muestra carga, vacío y error sin insertar HTML recibido.
- Las pruebas y el JAR se ejecutan desde terminal.

2. Qué significa MVC en Spring Web

MVC separa el Model, la View y el Controller. El Model representa información de la aplicación que una vista puede utilizar; no es sinónimo de una tabla ni de una entidad JPA. La View presenta esa información. El Controller traduce una solicitud HTTP en una acción, prepara una respuesta y delega la lógica que no pertenece al transporte.

Spring MVC implementa este enfoque sobre la pila Servlet. Tomcat recibe la solicitud y el DispatcherServlet actúa como controlador frontal. Los mapeos encuentran el método Java adecuado. Los resolutores de argumentos convierten parámetros; el método ejecuta su trabajo; Spring procesa su resultado.

Flujo de la consulta del catálogo

- El navegador solicita GET /api/v1/cursos?q=Spring.
- DispatcherServlet encuentra CourseController.listar y enlaza q.
- CourseController valida el límite y llama a CourseService.
- CourseService devuelve objetos CourseDto.
- Un HttpResponseMessageConverter serializa la lista como JSON.
- JavaScript presenta los datos en la lista del documento HTML.

En un MVC con plantillas, el controlador puede agregar atributos a Model y devolver un nombre de vista. Un ViewResolver localiza la plantilla y esta genera HTML en el servidor. En esta semana la rama JSON no resuelve una plantilla: se estudia el mismo despacho de Spring MVC, pero la representación se construye en el navegador. No llamaremos a esto renderizado Thymeleaf.

Qué debemos evitar

No genere HTML concatenando entradas del usuario en Java. No coloque la búsqueda en el método main. No añada @EnableWebMvc para un proyecto Boot estándar: puede reemplazar su configuración automática. Un DTO de respuesta no necesita @Entity ni una base de datos.

3. Rutas parámetros y tipos de respuesta

Elemento	Significado	Uso en TechLab
@Controller	Componente web con resolución de vistas o navegación.	PortalController
@RestController	@Controller más @ResponseBody.	CourseController
@RequestMapping	Base compartida de ruta y condiciones.	/api/v1/cursos
@GetMapping	Solo atiende GET.	Consulta sin mutaciones
@RequestParam	Enlaza un parámetro de consulta o formulario.	q=Spring
@PathVariable	Enlaza un segmento de la ruta.	/cursos/2
@Service	Registra lógica de aplicación como bean.	Búsqueda en memoria

Un String devuelto por @Controller suele ser un nombre de vista. El prefijo redirect: produce una nueva solicitud del navegador. En @RestController, un String normalmente se escribe en el cuerpo, no selecciona una vista. Esta distinción explica por qué PortalController no usa @RestController.

Solicitud	Resultado esperado
GET /portal	302 y Location /catalogo.html; luego el navegador recibe HTML.
GET /api/v1/cursos	200 y lista de tres cursos.
GET /api/v1/cursos?q=Spring	200 y un curso.
GET /api/v1/cursos?q=ZZZ	200 y []; vacío no es un error.
GET /api/v1/cursos/999	404 porque no existe.
GET /api/v1/cursos/abc	400 porque abc no se convierte a long.
POST /api/v1/cursos	405 porque no se implementa POST.

GET no debe grabar datos. Un parámetro q ausente toma el valor vacío. Las respuestas de error pueden variar en su cuerpo según la configuración; las pruebas comprobarán el estado HTTP y no dependerán de mensajes internos.

4. Preparación y estructura del proyecto

- Recupere TechLab de la semana 5 y haga un commit o una copia antes de cambiarlo.
- Valide `java --version` y el runtime del IDE en Java 25.
- Ejecute el test de contexto existente y abra `/actuator/health`.
- Confirme que `index.html` y los recursos del APF1 están bajo `static`.
- Abra la carpeta que contiene `pom.xml` y los comandos `mvnw`.

No genere otro proyecto ni cambie el paquete raíz. La clase `TechLabWebApplication` y `StartupReporter` de la semana 5 permanecen. Las nuevas clases deben estar bajo `pe.edu.utp.techlab` para que el escaneo encuentre los componentes.

Ruta relativa a <code>src/main</code>	Archivo nuevo
<code>java/pe/edu/utp/techlab/dto</code>	<code>CourseDto.java</code>
<code>java/pe/edu/utp/techlab/service</code>	<code>CourseService.java</code>
<code>java/pe/edu/utp/techlab/controller</code>	<code>CourseController.java</code> y <code>PortalController.java</code>
<code>resources/static</code>	<code>catalogo.html</code>
<code>resources/static/js</code>	<code>catalogo.js</code>

Las pruebas se crean en `src/test/java/pe/edu/utp/techlab/service` y `controller`, respectivamente. No coloque el README, las capturas ni el JAR dentro de `static`, porque esa carpeta es pública.

`src/main/resources/application.properties` completo

```
spring.application.name=techlab-web
server.port=8080
management.endpoints.web.exposure.include=health
management.endpoint.health.show-details=never
```

Mantenga `application-local.properties` de la semana 5 con `server.port=8081`. Para esta práctica ejecute sin el perfil local, de manera que los casos manuales usen el puerto 8080. La versión completa del POM aparece en la página siguiente como referencia verificable.

5. Dependencias completas de referencia

Conserve el wrapper. El POM siguiente contiene lo necesario para esta práctica y el test de contexto; no fuerza versiones de Jackson, Spring Framework ni Tomcat. El starter de pruebas de Actuator no es necesario porque aquí no escribiremos pruebas específicas de ese módulo.

pom.xml completo

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>
    <relativePath/>
  </parent>
  <groupId>pe.edu.utp</groupId>
  <artifactId>techlab-web</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <properties><java.version>25</java.version></properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-devtools</artifactId>
      <scope>runtime</scope> <optional>true</optional>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-webmvc-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <build><plugins><plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
  </plugin></plugins></build>
</project>
```

6. Crear el DTO y el servicio

Cree primero los paquetes dto y service. Un record expresa un portador de datos inmutable con constructor y accesores generados; titulo() y horas() son métodos Java, mientras que en JSON los campos son titulo y horas. Los ID son estables y no corresponden a filas de una base de datos.

src/main/java/pe/edu/utp/techlab/dto/CourseDto.java completo

```
package pe.edu.utp.techlab.dto;

public record CourseDto(long id, String titulo, int horas) {
}
```

src/main/java/pe/edu/utp/techlab/service/CourseService.java completo

```
package pe.edu.utp.techlab.service;

import java.util.List;
import java.util.Locale;
import java.util.Optional;
import org.springframework.stereotype.Service;
import pe.edu.utp.techlab.dto.CourseDto;

@Service
public class CourseService {
    private final List<CourseDto> cursos = List.of(
        new CourseDto(1L, "HTML y CSS", 12),
        new CourseDto(2L, "Bootstrap", 16),
        new CourseDto(3L, "Spring Boot", 20)
    );

    public List<CourseDto> buscar(String consulta) {
        String q = consulta.strip().toLowerCase(Locale.ROOT);
        return cursos.stream()
            .filter(c -> c.titulo().toLowerCase(Locale.ROOT)
                .contains(q))
            .toList();
    }

    public Optional<CourseDto> buscarPorId(long id) {
        return cursos.stream()
            .filter(c -> c.id() == id)
            .findFirst();
    }
}
```

List.of evita mutaciones. strip y Locale.ROOT normalizan q; una consulta vacía devuelve todos. Optional expresa ausencia sin null. El servicio no depende de HTTP y puede probarse sin arrancar Spring.

7. Implementar el controlador de consulta

Cree controller y el archivo siguiente. El constructor declara la dependencia; al existir un solo constructor no necesita @Autowired. El controlador solo adapta entradas y salidas HTTP. El límite de q se verifica también en el servidor, aunque HTML tenga maxlength.

src/main/java/pe/edu/utp/techlab/controller/CourseController.java completo

```
package pe.edu.utp.techlab.controller;

import java.util.List;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;
import pe.edu.utp.techlab.dto.CourseDto;
import pe.edu.utp.techlab.service.CourseService;

@RestController
@RequestMapping(value = "/api/v1/cursos",
    produces = "application/json")
public class CourseController {
    private final CourseService service;

    public CourseController(CourseService service) {
        this.service = service;
    }

    @GetMapping
    public List<CourseDto> listar(
        @RequestParam(name = "q", defaultValue = "") String q) {
        if (q.length() > 60) {
            throw new ResponseStatusException(
                HttpStatus.BAD_REQUEST, "Consulta demasiado larga");
        }
        return service.buscar(q);
    }

    @GetMapping("/{id}")
    public CourseDto detalle(@PathVariable("id") long id) {
        return service.buscarPorId(id).orElseThrow(() ->
            new ResponseStatusException(HttpStatus.NOT_FOUND,
                "Curso no encontrado"));
    }
}
```

8. Implementar navegación y comprobar las rutas

src/main/java/pe/edu/utp/techlab/controller/PortalController.java completo

```
package pe.edu.utp.techlab.controller;
```

```
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
```

```
@Controller
public class PortalController {
    @GetMapping("/portal")
    public String catalogo() {
        return "redirect:/catalogo.html";
    }

    @GetMapping("/inicio")
    public String inicio() {
        return "redirect:/";
    }
}
```

La redirección es interna y fija. No acepte un destino arbitrario del usuario, pues puede convertirse en una redirección abierta. La ruta /inicio devuelve al index.html del APF1 mediante la página de bienvenida de Boot. No mapee / si quiere conservar ese comportamiento automático.

Arranque desde la raíz del proyecto

```
# Windows PowerShell
.\mvnw.cmd spring-boot:run
```

```
# macOS o Linux
./mvnw spring-boot:run
```

- Abra /api/v1/cursos y confirme tres objetos con id, título y horas.
- Abra /api/v1/cursos?q=Spring y confirme un único resultado.
- Abra /api/v1/cursos/999 y revise 404 en DevTools Network.
- La ruta /portal todavía no puede completar su destino hasta crear catalogo.html.
- No añada CORS para estas solicitudes: HTML y API estarán en el mismo origen.

Evidencia de control. Registre ruta, método, estado y cuerpo. Si obtiene 404 en todas las rutas, revise paquete raíz, anotaciones y reinicio. Si la aplicación no inicia por un mapeo ambiguo, busque otro controlador con la misma combinación de método y ruta.

9. Crear la vista estática del catálogo

Cree un archivo adicional, sin reemplazar index.html del APF1. La vista utiliza Bootstrap ya trabajado. El ejemplo completo no requiere el bundle JavaScript de Bootstrap porque no contiene componentes interactivos del framework.

src/main/resources/static/catalogo.html completo

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Catálogo de cursos TechLab</title>
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css">
  <script src="js/catalogo.js" defer></script>
</head>
<body>
  <header class="bg-dark p-3">
    <nav class="container" aria-label="Navegación principal">
      <a class="text-white" href="/inicio">Inicio TechLab</a>
    </nav>
  </header>
  <main class="container py-4">
    <h1>Catálogo de cursos</h1>
    <p>Los datos de este catálogo provienen del servidor Java.</p>
    <form id="busqueda" class="mb-4" role="search">
      <label for="consulta" class="form-label">Buscar por título</label>
      <input id="consulta" name="q" type="search" maxlength="60"
class="form-control" aria-describedby="ayuda">
      <p id="ayuda" class="form-text">Hasta 60 caracteres.</p>
      <button id="buscar" type="submit" class="btn btn-primary">
        Buscar
      </button>
    </form>
    <p id="estado" role="status" aria-live="polite"></p>
    <ul id="cursos" class="list-group" aria-label="Cursos disponibles">
    </ul>
    <noscript>Active JavaScript para consultar el catálogo.</noscript>
  </main>
</body>
</html>
```

La etiqueta label está asociada al input y el estado se anuncia mediante aria-live. El script defer se ejecuta después de analizar el HTML. Si el CDN no está disponible, la página conserva contenido semántico aunque pierda estilos. Para una entrega sin internet, use la distribución local fijada de Bootstrap.

10. Presentar la respuesta sin insertar HTML

El siguiente archivo se muestra en dos partes consecutivas por legibilidad. Copie ambas, en orden, dentro de un único `catalogo.js`. No agregue separadores, no repita declaraciones y no omita ninguna línea. Esta primera parte prepara el DOM y construye la lista.

`src/main/resources/static/js/catalogo.js` parte 1 de 2

```
"use strict";

const formulario = document.querySelector("#busqueda");
const consulta = document.querySelector("#consulta");
const boton = document.querySelector("#buscar");
const lista = document.querySelector("#cursos");
const estado = document.querySelector("#estado");

function mostrarCursos(cursos) {
  lista.replaceChildren();
  for (const curso of cursos) {
    const item = document.createElement("li");
    item.className = "list-group-item";
    const titulo = document.createElement("h2");
    titulo.className = "h5";
    titulo.textContent = curso.titulo;
    const detalle = document.createElement("p");
    detalle.className = "mb-1";
    detalle.textContent = `${curso.horas} horas`;
    const enlace = document.createElement("a");
    enlace.href = `/api/v1/cursos/${encodeURIComponent(curso.id)}`;
    enlace.textContent = `Ver datos de ${curso.titulo}`;
    item.append(titulo, detalle, enlace);
    lista.append(item);
  }
}
```

`replaceChildren` elimina resultados anteriores. `createElement` crea nodos reales y `textContent` inserta texto sin interpretarlo como HTML. Así, un título no se convierte en una etiqueta ejecutable. `encodeURIComponent` protege el segmento generado; los ID de la respuesta son numéricos.

El enlace de detalle muestra JSON de manera intencional. Una página de detalle renderizada con plantilla se desarrollará cuando se incorpore Thymeleaf. No invente todavía un archivo HTML distinto por cada curso.

11. Consultar el servidor y manejar estados

catalogo.js parte 2 de 2 continuación inmediata

```
async function cargarCursos() {
  boton.disabled = true;
  formulario.setAttribute("aria-busy", "true");
  estado.textContent = "Consultando cursos...";
  lista.replaceChildren();
  try {
    const parametros = new URLSearchParams({q: consulta.value});
    const respuesta = await fetch(`/api/v1/cursos?${parametros}`, {
      headers: {Accept: "application/json"}
    });
    if (!respuesta.ok) {
      throw new Error(`HTTP ${respuesta.status}`);
    }
    const cursos = await respuesta.json();
    if (!Array.isArray(cursos)) {
      throw new Error("Formato de respuesta inesperado");
    }
    mostrarCursos(cursos);
    estado.textContent = cursos.length
      ? `${cursos.length} cursos encontrados.`
      : "No hay cursos que coincidan.";
  } catch (error) {
    estado.textContent = "No se pudo cargar el catálogo. Reintente.";
    console.error("Fallo al consultar el catálogo", error);
  } finally {
    boton.disabled = false;
    formulario.removeAttribute("aria-busy");
  }
}
```

```
formulario.addEventListener("submit", (evento) => {
  evento.preventDefault();
  if (!boton.disabled) cargarCursos();
});
cargarCursos();
```

fetch no rechaza automáticamente un 404 o 500; por eso se revisa respuesta.ok antes de leer JSON.

URLSearchParams codifica q. El botón deshabilitado evita consultas simultáneas desde el formulario. finally restablece el control incluso si hay un error. No se cambia el foco del teclado para anunciar resultados.

12. Integrar y depurar el catálogo

- Guarde los archivos e inicie Spring Boot. Abra `http://localhost:8080/portal`.
- En Network observe primero 302 de `/portal` y luego 200 de `catalogo.html`.
- Confirme que `catalogo.js` y la consulta de cursos reciben 200.
- Busque Spring, luego ZZZ y finalmente deje vacío el campo.
- Repita con teclado y con un ancho de 360 px.
- Detenga el servidor y envíe una nueva búsqueda para observar el error. Reinicie y reintente.
- Vuelva al APF1 mediante Inicio TechLab y compruebe navbar, modal y formulario.

No use Live Server en esta comprobación. Un puerto distinto es otro origen, y una URL relativa `/api/v1/cursos` apuntaría al servidor equivocado. Tampoco abra `catalogo.html` con `file://`; siempre use la aplicación Java.

Síntoma	Comprobación y corrección
JSON en vez de redirección	PortalController debe tener @Controller y devolver <code>redirect::</code> .
Error al resolver plantilla	Un String en @Controller sin prefijo se interpreta como nombre de vista.
La búsqueda devuelve todos	Observe q en Network y verifique <code>URLSearchParams</code> .
404 de <code>catalogo.js</code>	Compare <code>js/catalogo.js</code> con la ubicación y las mayúsculas.
Failed to fetch	Revise servidor, puerto, origen y conectividad antes de añadir CORS.
Unexpected token al leer JSON	Observe cuerpo y tipo de respuesta; puede haberse recibido HTML de error.
Bean <code>CourseService</code> no encontrado	Confirme @Service y ubicación bajo el paquete raíz.

Evidencia de control. Capture la lista filtrada y Network. Una captura visual sin la solicitud no demuestra que los datos provengan de Java. El servidor puede entregar datos dinámicos a un archivo HTML estático; son conceptos diferentes.

13. Probar la lógica sin Spring

src/test/java/pe/edu/utp/techlab/service/CourseServiceTests.java completo

```
package pe.edu.utp.techlab.service;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CourseServiceTests {
    private final CourseService service = new CourseService();

    @Test
    void consultaVaciaDevuelveTodos() {
        assertEquals(3, service.buscar("").size());
    }

    @Test
    void busquedaNormalizaEspaciosYMayusculas() {
        var resultado = service.buscar(" SPRING ");
        assertEquals(1, resultado.size());
        assertEquals(3L, resultado.getFirst().id());
    }

    @Test
    void idDesconocidoNoExiste() {
        assertTrue(service.buscarPorId(999L).isEmpty());
    }
}
```

Esta prueba usa JUnit sin @SpringBootTest. new CourseService es correcto aquí porque se está probando una clase aislada, no creando dependencias manualmente dentro de la aplicación. Los tres casos verifican listado, normalización y ausencia.

Ejecutar pruebas desde la raíz

```
# Windows
.\mvnw.cmd test
# macOS o Linux
./mvnw test
```

Mantenga TechLabWebApplicationTests de la semana 5. Ese test confirma que el contexto se crea; los nuevos tests comprueban comportamientos concretos. Nunca use -DskipTests para ocultar fallos en la evidencia final.

14. Probar el transporte con MockMvc

MockMvc ejercita Spring MVC sin abrir un puerto de red real. @WebMvcTest carga una porción web del contexto. @Import registra el servicio real y su conjunto de datos determinista. El test se presenta completo en dos partes para copiar en un único archivo.

src/test/java/pe/edu/utp/techlab/controller/WebRoutesTests.java parte 1 de 2

```
package pe.edu.utp.techlab.controller;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Import;
import org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest;
import org.springframework.test.web.servlet.MockMvc;
import pe.edu.utp.techlab.service.CourseService;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest({CourseController.class, PortalController.class})
@Import(CourseService.class)
class WebRoutesTests {
    @Autowired
    private MockMvc mvc;

    @Test
    void listadoYFiltroFuncionan() throws Exception {
        mvc.perform(get("/api/v1/cursos"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.length()").value(3));
        mvc.perform(get("/api/v1/cursos").param("q", "SPRING"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$[0].id").value(3));
    }
}
```

En Spring Boot 4.1 el import de WebMvcTest pertenece a org.springframework.boot.webmvc.test.autoconfigure. No copie el paquete antiguo de tutoriales de Boot 3. Las importaciones estáticas permiten leer las expectativas como una secuencia de solicitud y resultado.

El primer test comprueba la cantidad y el filtro. jsonPath se aplica a la respuesta JSON, no al record Java. No se usa @MockitoBean porque esta práctica prueba el servicio real en la capa web.

15. Comprobar errores y redirecciones

WebRoutesTests.java parte 2 de 2 continuación inmediata

```
@Test
void detalleYErroresUsanEstadosHttp() throws Exception {
    mvc.perform(get("/api/v1/cursos/2"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.titulo").value("Bootstrap"));
    mvc.perform(get("/api/v1/cursos/999"))
        .andExpect(status().isNotFound());
    mvc.perform(get("/api/v1/cursos/abc"))
        .andExpect(status().isBadRequest());
    mvc.perform(get("/api/v1/cursos").param("q", "x".repeat(61)))
        .andExpect(status().isBadRequest());
}

@Test
void metodoNoPermitidoYRedireccion() throws Exception {
    mvc.perform(post("/api/v1/cursos"))
        .andExpect(status().isMethodNotAllowed());
    mvc.perform(get("/portal"))
        .andExpect(status().isFound())
        .andExpect(redirectedUrl("/catalogo.html"));
}
}
```

El 400 de abc surge de la conversión de argumentos de Spring. El 400 de una consulta larga lo decide el controlador. El 404 refleja ausencia y el 405 distingue una ruta existente de un método no implementado. La prueba de redirección verifica Location, pero no renderiza el catálogo ni ejecuta JavaScript.

Ejecute todos los tests y registre el resumen real. Para probar presentación, accesibilidad, fetch y recursos estáticos todavía necesita el navegador. MockMvc no demuestra el funcionamiento de un JAR ejecutado en Tomcat.

16. Matriz de pruebas y empaquetado

ID	Procedimiento	Esperado
01	GET /portal sin seguir redirección	302 Location /catalogo.html
02	Abrir /portal en navegador	Catálogo y tres cursos
03	Buscar SPRING con espacios	Un curso ID 3
04	Buscar ZZZ	200 lista vacía y aviso
05	GET /api/v1/cursos/2	200 Bootstrap
06	GET /api/v1/cursos/999	404
07	GET /api/v1/cursos/abc	400
08	GET con q de 61 caracteres	400
09	POST /api/v1/cursos	405
10	Detener servidor y buscar	Mensaje de error y botón habilitado
11	Recorrer teclado y 360 px	Controles usables sin desborde
12	Ejecutar test y clean package	BUILD SUCCESS y JAR nuevo
13	Ejecutar JAR y repetir catálogo	Mismas rutas y datos
14	Regresión del APF1 y health	Funciones anteriores y UP

Registre para cada ID resultado real, pasa o falla, corrección aplicada y evidencia. No marque pasa por el resultado esperado; repita el caso después de corregir. Para los errores de límite y método utilice una herramienta HTTP o los tests automatizados, no el formulario HTML.

Empaquetar y ejecutar después de detener el proceso anterior

```
# Windows
.\mvnw.cmd clean package
# macOS o Linux
./mvnw clean package
# Todos los sistemas
java -jar target/techlab-web-0.0.1-SNAPSHOT.jar
```

17. Reto y buenas prácticas

Reto de transferencia

Implemente GET `/api/v1/cursos/{id}/duracion` que devuelva un nuevo record con id y horas. Mantenga la consulta en el servicio; no duplique la lista ni escriba condiciones por cada ID. Para un curso ausente devuelva 404 y para un ID no numérico conserve 400.

- Cree DurationDto en dto con long id e int horas.
- Añada el método al controlador con `@PathVariable` explícito.
- Reutilice `buscarPorId` y convierta el resultado al nuevo DTO.
- Añada pruebas para ID 2, ID 999 e ID abc.
- Documente JSON esperado y resultados reales en el README.
- Explique por qué `/id/duracion` no entra en conflicto con `/id`.

Criterio de aceptación. `/api/v1/cursos/2/duracion` devuelve 200 con `{"id":2,"horas":16}`; la ruta no altera los datos y las pruebas previas siguen pasando. El reto no exige persistencia ni cambios en el catálogo HTML.

Límites de seguridad y operación

La aplicación del laboratorio es local y pública: no representa un servicio listo para producción. No coloque datos personales, credenciales o capturas privadas en las respuestas. Mantenga solo health expuesto y detalles ocultos. Validación de entrada no equivale a autorización.

Evite innerHTML para los datos recibidos. No solucione un error de origen con CORS abierto. Mantenga versiones fijadas, dependencias administradas y archivos de build fuera de Git. El servicio usa datos inmutables y no guarda estado de cada solicitud en campos compartidos.

Preparación para la semana 7

Conserve las capas y sus pruebas. En la siguiente práctica se podrá reutilizar el servicio para agregar atributos a Model y renderizar plantillas Thymeleaf. Compararemos HTML generado en el servidor con la presentación actual mediante fetch sin confundir ambos flujos.

18. Comprobación conceptual y referencias

- ¿Qué responsabilidad cumple DispatcherServlet antes del controlador?
- ¿Por qué un JSON servido por Spring MVC no es una plantilla Thymeleaf?
- ¿Qué diferencia existe entre Model, DTO y entidad de persistencia?
- ¿Cómo cambia el significado de String entre Controller y RestController?
- ¿Cuándo conviene RequestParam y cuándo PathVariable?
- ¿Por qué una búsqueda sin coincidencias responde 200 y no 404?
- ¿Qué produce 400 antes de ejecutar detalle y qué caso decide el controlador?
- ¿Por qué fetch requiere revisar ok además de usar catch?
- ¿Qué demuestra MockMvc y qué debe comprobar el navegador?
- ¿Cómo reutilizará CourseService al incorporar vistas dinámicas?

Referencias oficiales

[Spring Boot requisitos y compatibilidad](#)

[Spring MVC mapeo de solicitudes](#)

[Spring MVC parámetros de solicitud](#)

[Spring MVC cuerpo de respuesta](#)

[Spring Framework MockMvc](#)

[Spring Boot WebMvcTest API](#)

[Spring Boot aplicaciones Servlet](#)

[MDN uso de Fetch](#)